FPT Education

TRƯỜNG ĐẠI HỌC FPT

Module 6 - Responsive UI & Adaptive Layouts

Contents

- What Responsive UI means
- Screen sizes, device types, and safe areas
- MediaQuery for screen information
- LayoutBuilder for constraint-based rendering
- OrientationBuilder for rotation
- Responsive widgets (Expanded, Wrap, Flexible)
- Building a real case study: Movie Genre Screen
- How to test responsive UI on multiple devices

Why Responsive UI Matters

Flutter runs on:

- Small phones
- Large phones
- Tablets
- Foldables
- Desktop browsers

Without responsive design:

- Overflow errors
- Misaligned widgets
- Text clipping
- Bad user experience

Responsive vs Adaptive UI

Responsive UI: Layout changes depending on available space.
Examples:

- Column → Row
- Grid 2 columns → 4 columns
- Padding automatically adjusts based on screen width.
- Adaptive UI: Layout changes depending on device type.
  Examples:
  - Bottom Nav → NavigationRail on tablet
  - Drawer → Sidebar on desktop
    - *Code Snippet*

```
if (width < 600) return MobileView();
return TabletView();
```

MediaQuery: Getting Screen Size

MediaQuery allows you to access device and screen information, such as:

- Screen width
- Height
- Orientation
- Safe area padding
- Pixel density

Code demo:

```
final width = MediaQuery.of(context).size.width;

final height = MediaQuery.of(context).size.height;
```

Using Breakpoints

Breakpoints help define layout logic.

- Example Rule:
  - < 400px → Small layout
  - 400–800px → Medium layout
  - 800px → Large layout

```
if (width < 400) return SmallLayout();
else if (width < 800) return
MediumLayout();
return LargeLayout();
```

Code Demo (Runnable):

```
Widget build(BuildContext context) {
  final w =
MediaQuery.of(context).size.width;
  return Scaffold(
    body: Center(
      child: Text(
        w < 600 ? "Mobile Layout" : "Tablet
Layout",
        style: TextStyle(fontSize: 24),
      ),
    ),
  );
}
```

LayoutBuilder

- **LayoutBuilder gives parent constraints, not full screen size.**

Code Example:

```
LayoutBuilder(
  builder: (context, c) {
    return c.maxWidth > 500
      ? WideCard()
      : NarrowCard();
  },
);
```

Code example:

```
LayoutBuilder(
  builder: (context, c) {
    return Container(
      color: c.maxWidth > 300 ? Colors.blue
             : Colors.red,
      height: 120,
    );
  },
);
```

OrientationBuilder

- **Detects device rotation.**

```
OrientationBuilder(
  builder: (context, orientation) {
    return orientation == Orientation.portrait
      ? PortraitUI()
      : LandscapeUI();
  },
);
```

- **Demo:**
  - Portrait: list
  - Landscape: two-column layout

Responsive Widgets Overview

Useful widgets:

- Expanded – prevents overflow
- Flexible – controls how a child flexes within a Row or Column
- Wrap – auto line wrapping
- AspectRatio – avoids image stretching
- FractionallySizedBox – percent-based size
- GridView – multi-column responsive layout

Responsive GridView

- GridView allows building scrollable grids that automatically adapt to screen width by adjusting the number of columns.
- Code Example:

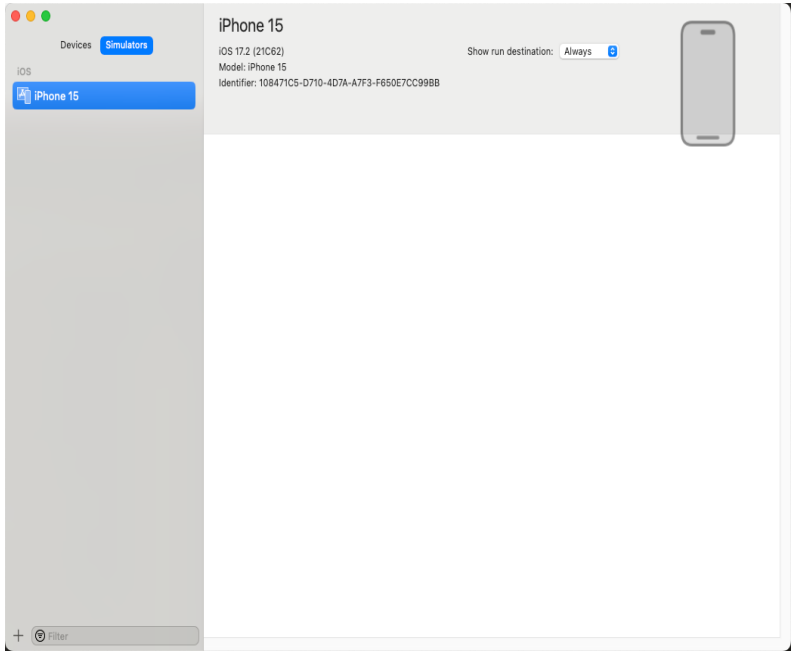
```
GridView.count(
        crossAxisCount: width < 600 ? 2 : 4,
);
```

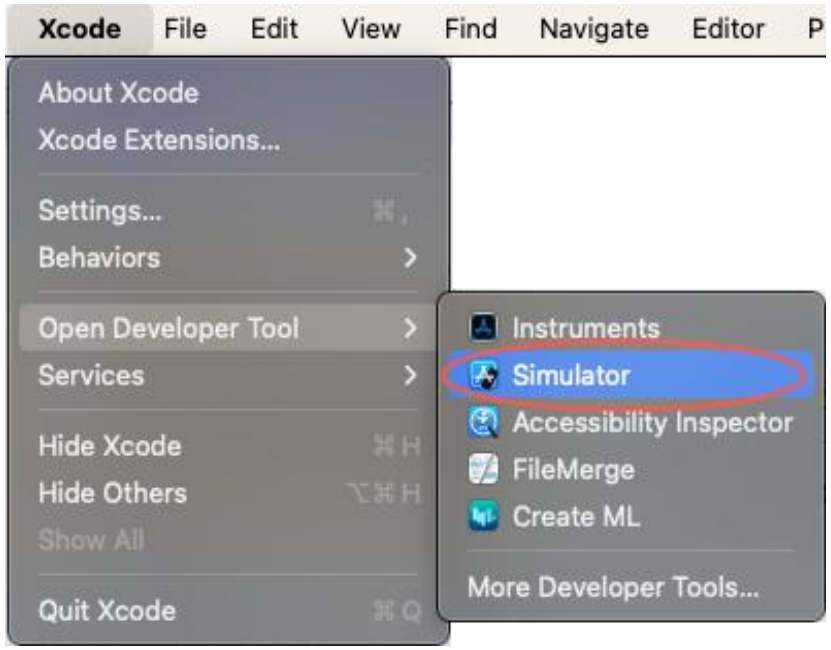
- Mobile: 2 columns
- Tablet: 3–4 columns
- Desktop: 5–6 columns

SafeArea & Modern Devices

- Dynamic Island / notches / curved screens
  → Some space cannot be used.
- Use:

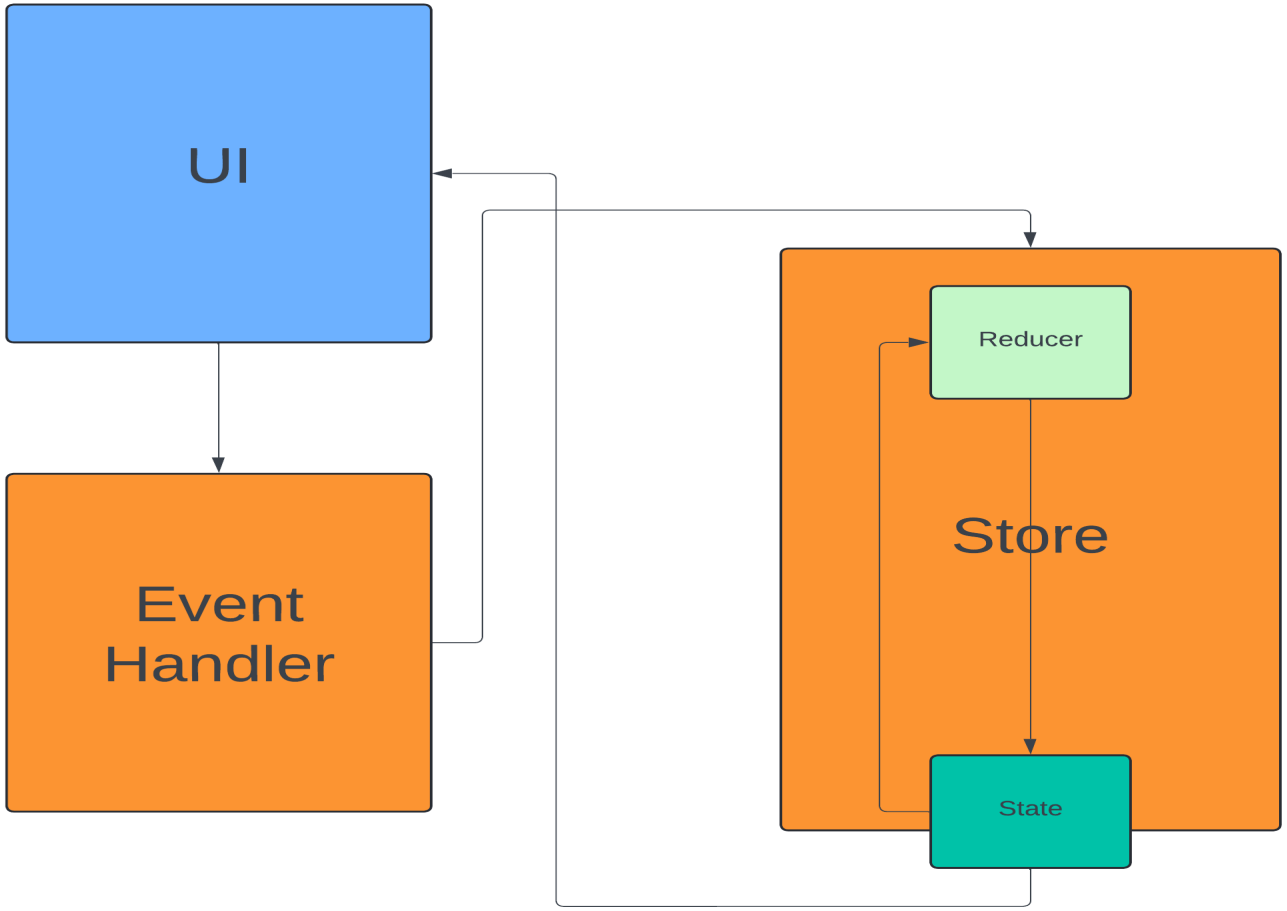
  SafeArea(child: ...)





Real Case Study: Genre Screen

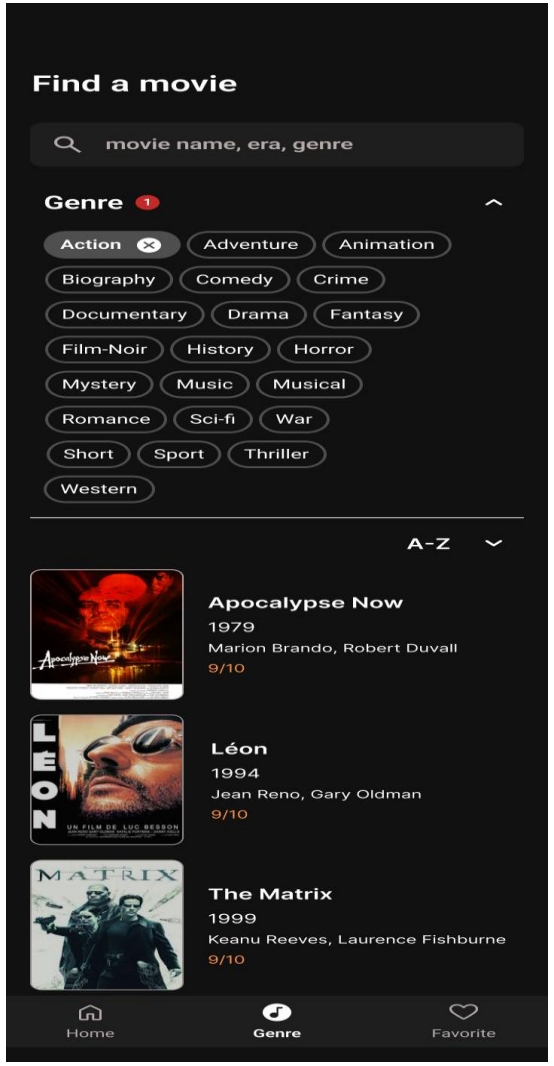
- **We now apply all concepts to a real UI**



Responsive Search Bar

- Key behaviors:
  - Full width
  - Padded for large screens
  - Wrapped in SafeArea
- **Mini Code:**

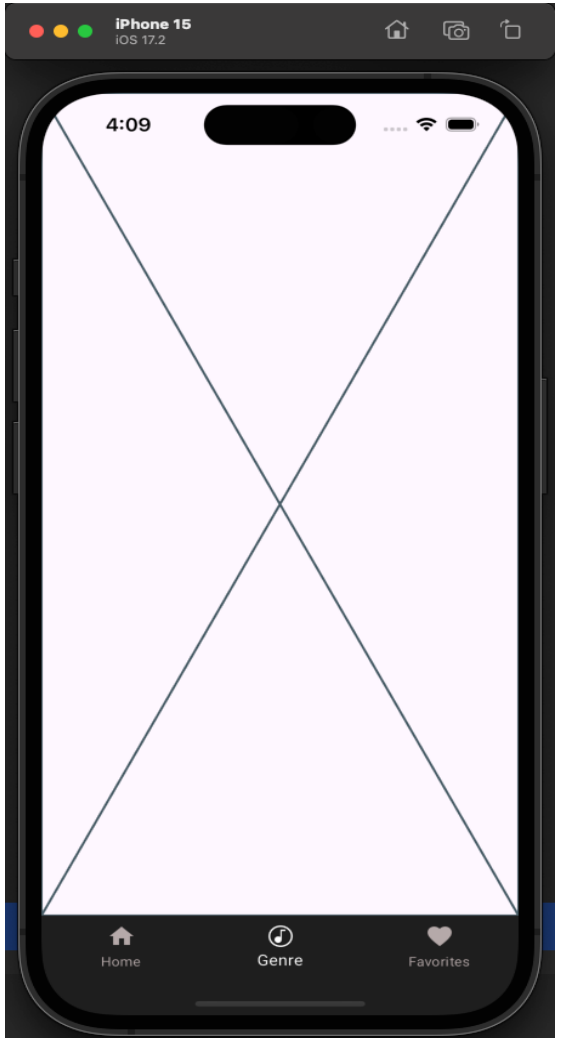
```
Container(
  padding:  EdgeInsets.symmetric(horizontal: 16),
  child: TextField(...),
)
```



Genre Chips (Collapsed State)

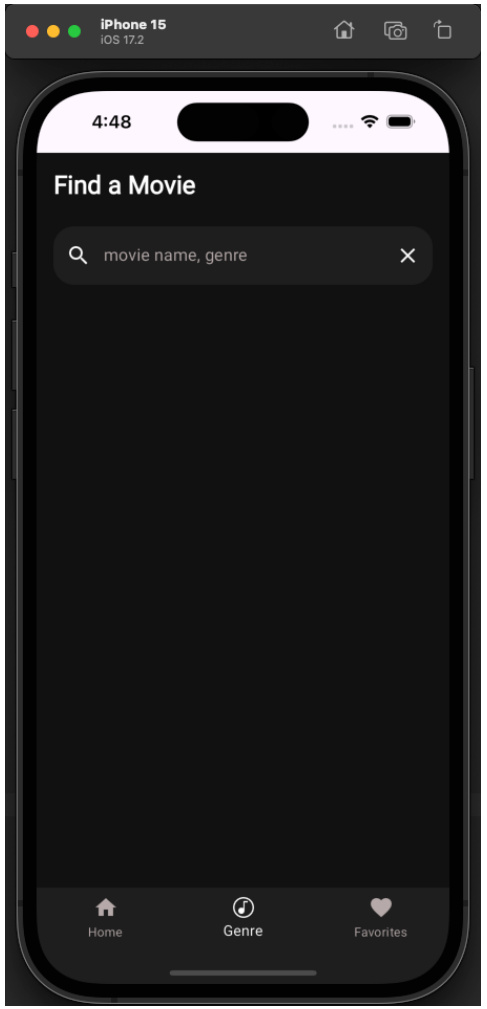
- Use the Wrap widget to ensure chips automatically flow to the next line when horizontal space is insufficient:

```
Wrap(
  spacing: 8,
  runSpacing: 8,
  children: genres.map(_buildChip).toList(),
)
```



Genre Chips (Expanded State)

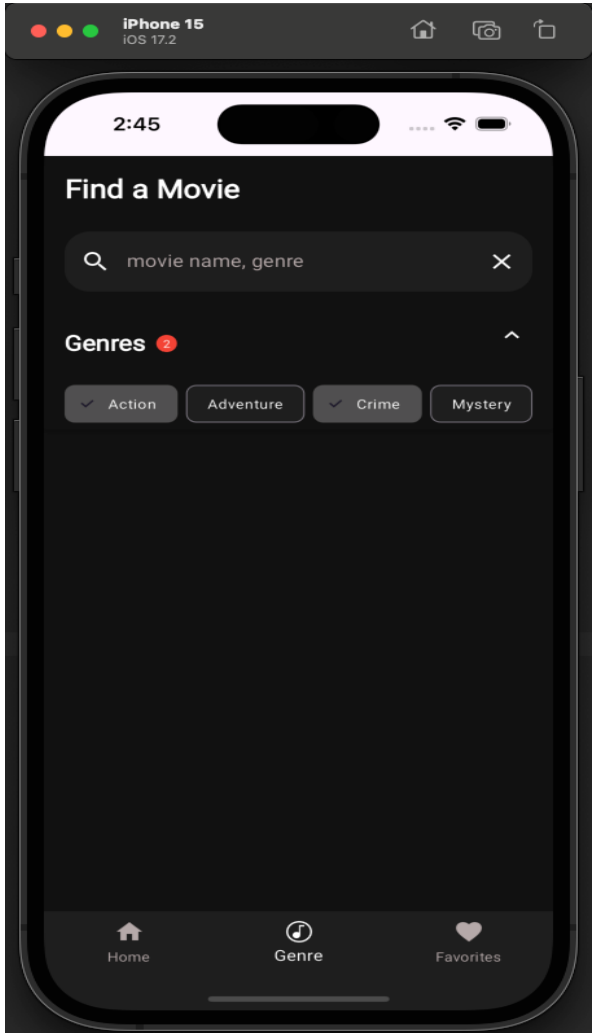
- Chips wrap automatically
- More space → more chips per row
- Less space → fewer chips per row
- This behavior is achieved automatically by using Wrap, without conditional logic.



Sort Dropdown

- Sorting options include A–Z, Z–A, Rating, and Year.
- Code Snippet:

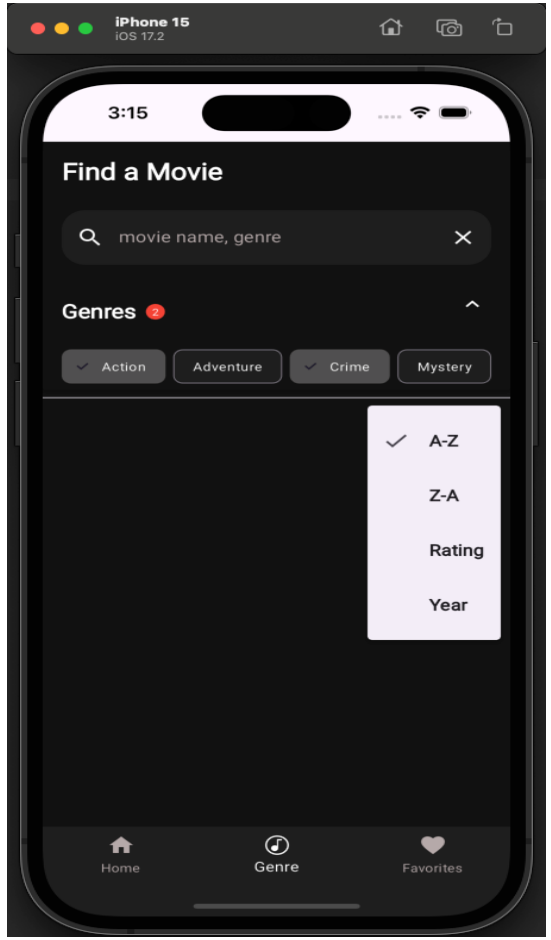
```
DropdownButton(
  value: selectedSort,
  items: [...],
  onChanged: (v) => setState(() =>
selectedSort = v!),
)
```



Responsive Movie Card

- **Code Snippet:**

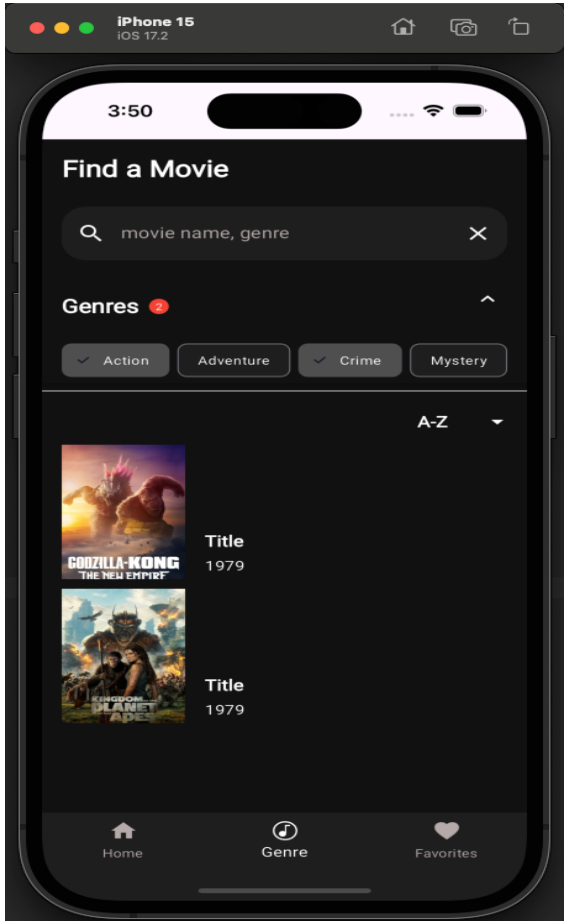
```
LayoutBuilder(
  builder: (context, c) {
    final isWide = c.maxWidth > 600;
    return Row(
      children: [
        Image.network(poster, width: isWide ? 150 : 100),
        SizedBox(width: 16),
        Expanded(child: Text(title))
      ],
    );
  },
)
```



Completed Responsive Screen

Elements included:

- Responsive search
- Dynamic chips
- Sort menu
- Responsive movie list



Lab 6.1

Responsive Hero Section

- Students must create a responsive home banner.
  - Width scaling
  - Height scaling
  - Breakpoints
  - Buttons adapt size

Lab 6.2

Rebuild the Genre Screen (Figures 6.1–6.9)

Requirements:

- Responsive search bar
- Wrap-based chips
- Sort dropdown
- Movie list responsive
- Test on phone + tablet sizes

Lab 6.3

Tablet Layout Enhancement

For screens > 800px:

- Add sidebar filter
- 2-column movie list
- Increased spacin

Best Practices

- Avoid fixed values
- Use constraints-aware components
- Prefer Wrap over Row for long chip lists
- Test often on multiple screen sizes
- Use Flutter Inspector's "Layout Explorer"

Summary

- Responsive vs Adaptive
- MediaQuery / LayoutBuilder / OrientationBuilder
- Responsive Widgets
- Implemented a full responsive Genre screen
- Applied GridView and Wrap techniques

References

- Mastering Flutter 2025 – Ch.5, 6
- docs.flutter.dev
- **https://dartpad.dev**